

Last-n Reasoning Patch

Research Progress; April 9th 2025

Overview

Goal: Build a minimal patching system optimized for reasoning models

Harness Crash → [Reasoning LLM System] → Patch

Using ARVO (OSSFuzz vulns w/ ground truth patches) as primary dataset

- Allows analysis of ground truth fixes
- Docker-based; allows easy patch, recompile, rerun on crashing input

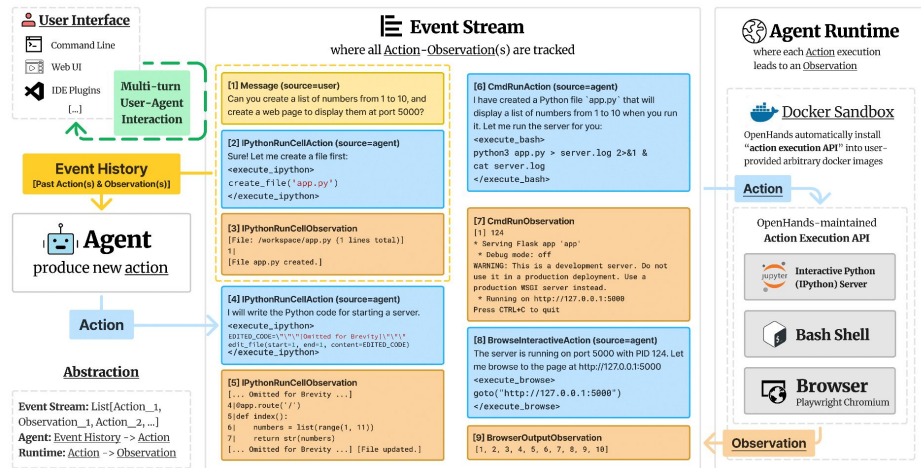
Current sota of LLM Patching

Problem: LLM cannot create patch when given raw crash dump and entire source code

- Context length exceeded failures
- LLM becomes too unfocused & worse at writing correct code

Existing Solutions:

- Agent-based systems to reverse program and debug issue
 - Built around interacting with codebase using tools in a patch-compile-test loop
 - Compiling in the loop is time-expensive
 - Built originally with non-reasoning LLM models in mind



Recent code-writing sota: OpenHands

Insight 1: Assertive Buggy Function

New Solution: Assert Function is Buggy to LLM

- Majority of ground-truth fixes only modify a few lines of code in a single function
- LLM Generations are parallelizable/ not time-expensive
- Empirical to verify result (recompile & rerun crashing input)
- Performs well with foundation LLM, and very well with reasoning LLMs

Prompt snippet:

```
...  
We know that the bug is in the code of the  
{target_name} function.  
Fix the bug by modifying only the code of this  
function.
```

Issue:

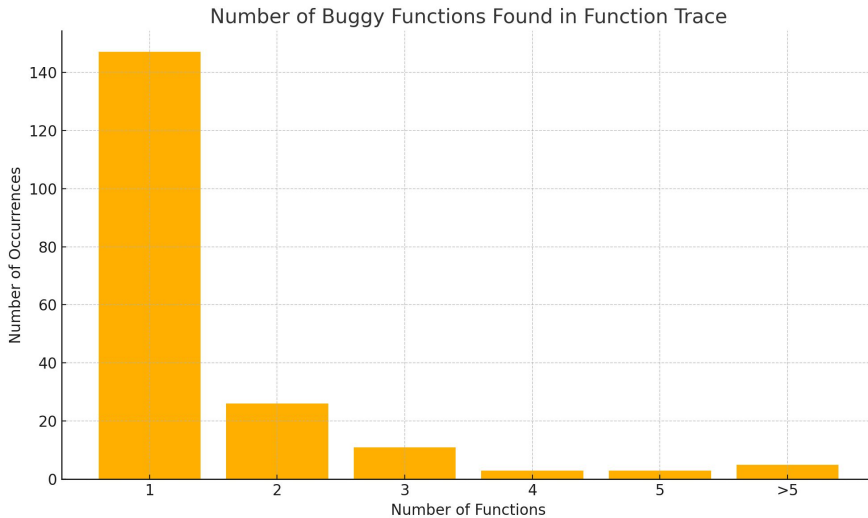
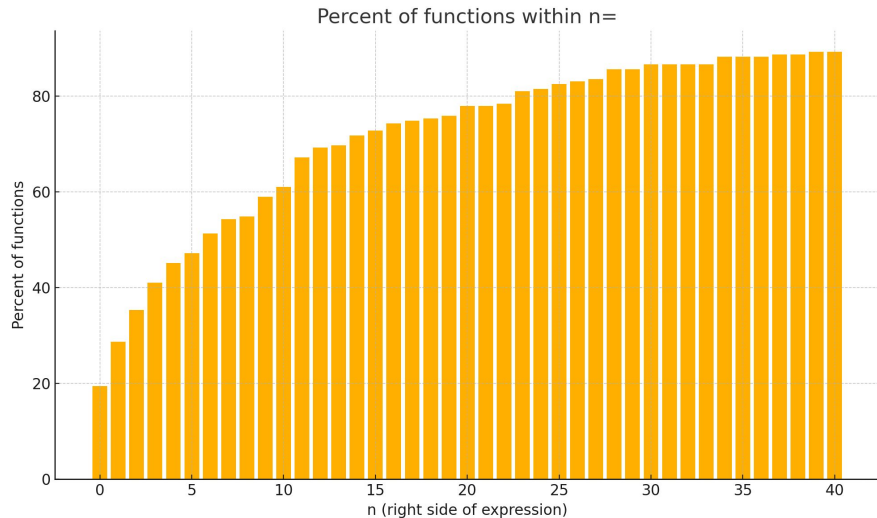
- Too many functions in source code; not viable to try on all of them

Insight 2: Buggy Function Heuristics

~88% of tcs had at least 1 buggy function in last 30 distinct functions from crash

- → using $n=30$ for experiments going forward

Most OSSFuzz crashes of C programs only execute 1 buggy function



Patching System

Given Crash Info from OSSFuzz/ ARVO:

1. PINtool: Extract Function Trace
 - a. Extract last 30 unique functions in trace
2. LLM: List Top 5 likely buggy functions
 - a. Prompt 1x w/ Context:
 - i. Harness crash info
 - ii. Function trace w/ last-n stats
3. LLM: Generate Patch (diff-like)
 - a. Prompt 6x per function w/ Context:
 - i. Harness crash info
 - ii. Hexdump of crashing input
 - iii. Function trace w/ last-n stats
 - iv. Source code for all called functions
4. LLM: Apply patch to source code
 - a. GitHub Copilot for now
5. ARVO: Recompile and run

Reasoning Model for Buggy Function Deduction

- 1x o1 (OpenAI, Dec 5 2024)

Reasoning Models for Patching

- 2x o1 (OpenAI, Dec 5 2024)
- 2x o3-mini high (OpenAI, Jan 31 2025)
- 2x Claude 3.7 Sonnet Extended (Anthropic, Feb 24 2025)

Prompt Samples in Appendix

Experiment + Results

Selected 26 tcs from ARVO where bug was limited to a single function (according to ground truth)

Covers:

- 23 distinct C ossfuzz projects
- 8 categories of crash types

Results:

Buggy function deduction:

- 19 Included Buggy Function (73.1%)
- 7 Not Included Buggy Function (26.9%)

Patch given correct buggy function oracle:

- 18 Patched (69.2%)
- 4 Not Patched (15.4%)
- 4 Not Generated (15.4%)

Effective End to End

- 14 Success (53.8%)
- 12 Failed (46.2%)

Limitations & Next Steps

- Better reasoning models have come out after experiment completed:
 - Gemini 2.5 Pro (Google, Mar 25 2025)
 - Reported sota performance on Code Editing tasks (Aider Polyglot benchmark)
 - 1 million token context window (vs o1:32k, o3-mini:200k, claude-3.7:200k)
 - Will allow programs with larger function and testcase lengths
- Improve Buggy Function Deduction Algorithm
 - Currently using very naive approach (1x attempt of top 5 from all functions called)
- Ablation study on Patching prompt components
 - Was able to improve BFD results significantly by minimizing prompt context
- Extend strategy for Multiple Buggy Functions

References

ARVO: Atlas of Reproducible Vulnerabilities for Open Source Software: <https://arxiv.org/pdf/2408.02153>

A Large-Scale Empirical Study of Security Patches: <https://dl.acm.org/doi/10.1145/3133956.3134072>

Aider LLM Leaderboards (Polyglot Leaderboard): <https://aider.chat/docs/leaderboards/#polyglot-leaderboard>

OpenHands: <https://arxiv.org/pdf/2406.01304>

Appendix: Prompts: Buggy Function Deduction

You are an expert security analyst. A fuzzer has found a vulnerability in the `{context.project}` project. Provided are the relevant artifacts you need to fix the bug.

Here is the crash metadata from the fuzzer:

```
{context.crash_metadata}
```

Here is a list of all of the distinct functions from the `{context.project}` source code that were called. The first number in each row corresponds to the total number of function calls you need to go back in order to get to the last time the function was invoked. The second number corresponds to the number of distinct functions you need to go back to see the last time the function was invoked

```
{context.last_n_all_functions.list_records()}
```

Here are the last 100 functions that we called, including duplicates.

```
{'\n'.join(context.last_100_calls)}
```

Reason through this bug and return the top 5 functions in the order of which you think most likely needs to be patched in order to fix the bug.

```
```output
```

```
1. name_of_most_likely_buggy_function
```

```
2. ...
```

```
3. ...
```

```
4. ...
```

```
5. name_of_least_likely_buggy_function
```

```
```
```

Format the output code block exactly as shown above. The function names should be the exact names of the functions in the source code.

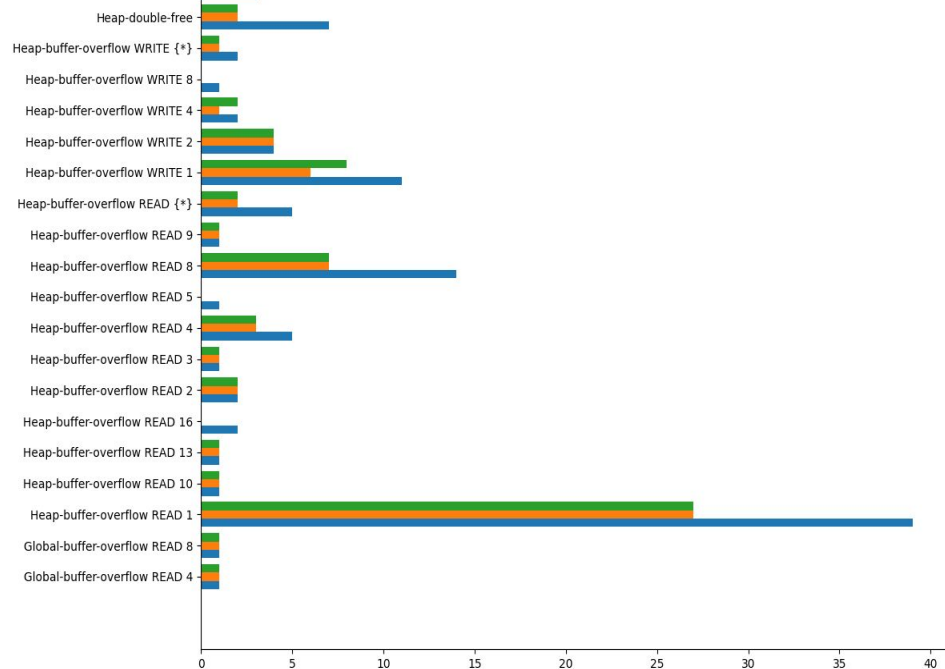
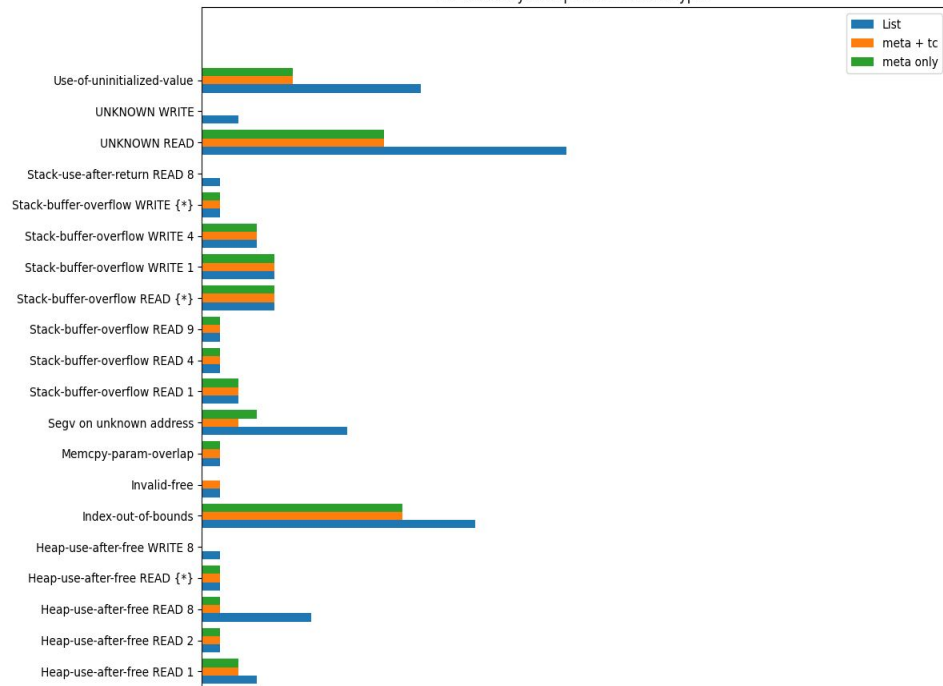
Appendix: Prompts: Assertive Function Patch

```
You are an expert security analyst. A fuzzer has found a vulnerability in the {context.project} project. Provided are the relevant artifacts you need to fix the bug.
Here is the crash metadata from the fuzzer:
{context.crash_metadata}
Here is the testcase that caused the crash:
{bytes_to_hexdump(context.fuzzer_testcase_bytes)}
Here is a list of all of the distinct functions from the {context.project} source code that were called. The first number in each row corresponds to the total number of function calls you need to go back in order to get to the last time the function was invoked. The second number corresponds to the number of distinct functions you need to go back to see the last time the function was invoked
{context.last_n_all_functions.list_records()}
Here is the source code for these functions:
{'\n----\n'.join([context.last_n_all_functions.source_code[fname] if fname in context.last_n_all_functions.source_code.keys() else f'Source code for {fname} not provided' for fname in [record[0] for record in context.last_n_all_functions.records]])}
Here are the last 100 functions that we called, including duplicates.
{'\n'.join(context.last_100_calls)}

Reason through this bug and output a patch formatted as
```diff
Diff content here
```
We know that the bug is in the code of the {target_name} function.
Fix the bug by modifying only the code of this function.
```

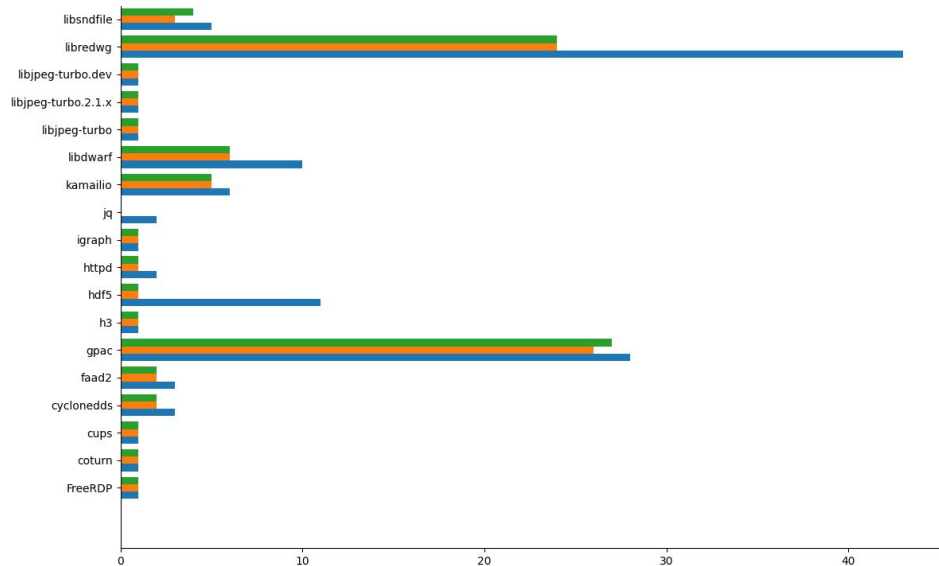
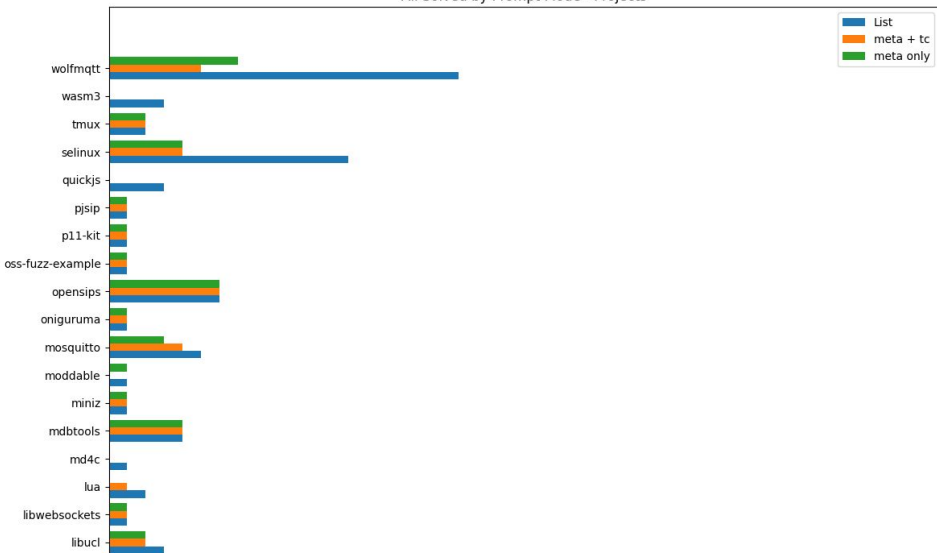
Appendix: BFD Results Crash Types Breakdown

All Solved by Prompt Mode - Crash Types



Appendix: BFD Results Project Types Breakdown

All Solved by Prompt Mode - Projects



Analysis

System does well in the following scenarios:

System does poorly in the following scenarios: